

PATRÓN DE DISEÑO: VISITANTE (VISITOR)

Capítulo 18 — Patrones de Diseño en Java

Traducción técnica al español

1. Descripción

Este patrón fue descrito previamente en GoF95.

El patrón Visitante es útil para diseñar una operación que se aplica sobre una colección de objetos de una jerarquía de clases. El patrón Visitante permite que la operación sea definida sin modificar la clase de ninguno de los objetos de dicha colección.

Para lograr esto, el patrón Visitante sugiere definir la operación en una clase separada denominada clase visitante. Esto desacopla la operación del conjunto de objetos sobre el que opera. Para cada nueva operación a definir, se crea una nueva clase visitante. Dado que la operación debe ejecutarse sobre un conjunto de objetos, el visitante necesita una forma de acceder a los miembros públicos de esos objetos. Este requisito puede abordarse implementando las siguientes ideas de diseño.

Idea de Diseño 1

Toda clase visitante que opera sobre objetos de un mismo conjunto de clases debe diseñarse para implementar una interfaz VisitorInterface correspondiente. La VisitorInterface declara un conjunto de métodos visit(TipoDeObjeto), uno por cada tipo de objeto de la colección. Cada uno de estos métodos se encarga de procesar instancias de una clase específica. Por ejemplo, si la colección de objetos contiene instancias de ClaseA y ClaseB, la interfaz Visitor declararía los dos métodos siguientes:

```
visit(ClaseA objClaseA)    // para procesar objetos de ClaseA
visit(ClaseB objClaseB)    // para procesar objetos de ClaseB
```

Cada objeto de la colección realiza una llamada al método visit(TipoDeObjeto) correspondiente, pasándose a sí mismo como argumento. El implementador (visitante) de la VisitorInterface puede acceder a la información requerida por la operación que implementa, accediendo a los miembros públicos (métodos y atributos) de la instancia del objeto que se le pasa mediante esa llamada al método.

Idea de Diseño 2

Las clases de objetos de la colección deben definir un método:

```
accept(visitor)
```

Un cliente interesado en ejecutar la operación visitante debe:

- Crear una instancia del implementador (visitante) de la interfaz Visitor diseñada para llevar a cabo la operación requerida.
- Crear la colección de objetos e invocar el método `accept(visitor)` sobre cada miembro de la colección, pasándole la instancia del visitante creada anteriormente.

Como parte de la implementación del método `accept(visitor)`, cada objeto de la colección invoca el método `visit(TipoDeObjeto)` sobre la instancia del visitante. Dentro del método `visit(TipoDeObjeto)`, el visitante recupera los datos necesarios de la colección de objetos para realizar la operación para la que fue diseñado.

2. Definición de Nuevas Operaciones sobre la Colección

Siempre que se deba definir una nueva operación sobre la colección de objetos, se debe crear una nueva clase visitante con la implementación para la nueva operación. La clase visitante debe implementar todos los métodos `visit(TipoDeObjeto)` declarados en la interfaz `VisitorInterface` para procesar los datos de los distintos tipos de objetos.

Con este diseño, definir una nueva operación no requiere modificar las clases de la colección de objetos.

3. Incorporación de Objetos de un Nuevo Tipo a la Colección

Cuando se deba agregar un nuevo tipo de objeto a la colección y éste deba ser considerado dentro del alcance de una operación visitante ya existente:

- La clase del objeto debe proveer un método similar a `accept(visitor)` y, como parte de su implementación, debe invocar el método `visit(TipoDeObjeto)` sobre el objeto visitante que recibe como argumento.
- Un método `visit(TipoDeObjeto)` correspondiente debe agregarse a la interfaz `VisitorInterface` y debe ser implementado por todas las clases visitantes concretas.

Esto significa que, para que objetos de una clase existente sean incorporados a la colección y sean considerados dentro del alcance de una operación visitante ya existente, dicha clase debe ser modificada para implementar un método similar a `accept(visitor)`, si es que no existe. Esto implica que, cuando el patrón Visitor se aplica por primera vez a una clase, se debe tener acceso a su código fuente o bien la clase debe ser subclassificada para agregar la implementación del método `accept(visitor)`.

4. Ejemplo

Diseñemos una aplicación para definir operaciones sobre una colección de objetos Order (Pedido). Los pedidos pueden ser de diferentes tipos. Consideramos los tres tipos de pedidos siguientes:

- Pedido al exterior (Overseas Order): pedido proveniente de países distintos a los EE.UU. Se cobra un cargo adicional por envío y manejo para este tipo de pedido.
- Pedido de California (California Order): pedido nacional con dirección de envío en California. Se aplica un impuesto de ventas adicional a este tipo de pedido.
- Pedido fuera de California (Non-California Order): pedido nacional con dirección de envío fuera de California. No aplica impuesto de ventas adicional.

4.1 Enfoque de Diseño I

Supongamos que deseamos definir una operación `getMaxCAOrderAmount` para encontrar el monto máximo de un pedido de California. Podemos definirla dentro de la clase `Order` como se ilustra en la Figura 18.1.

La clase `Order` mantiene una variable miembro estática `orderTotalCA` para llevar seguimiento del monto del pedido. Cada vez que se crea un nuevo pedido, la variable miembro se actualiza con el monto del pedido si el nuevo pedido es mayor que el total ya almacenado en `orderTotalCA`.

Si deseáramos agregar más métodos como `getMinCAOrderAmount`, `getCAOrderTotal`, `getMinNonCAOrderAmount`, `getOverseasOrderAmount`, etc., para obtener distintos tipos de totales de pedidos, el código de la clase `Order` debería modificarse para cada nueva operación. Como resultado, la clase puede volverse rápidamente recargada y difícil de mantener.

Figura 18.1 — Clase `Order` (Enfoque I)

Order
<pre>getMaxCAOrderAmount(): double orderTotalCA: double</pre>

4.2 Enfoque de Diseño II

Dado que los pedidos son de tres tipos distintos, podemos diseñar tres subclases de `Order`, cada una representando un tipo específico de pedido. Con este nuevo diseño, las operaciones relacionadas pueden mantenerse dentro de cada subclase de `Order`. Aunque esta estructura de clases es más manejable, aún presenta dos limitaciones:

- No existe un lugar apropiado donde definir operaciones que involucren diferentes tipos de pedidos.
- Cada vez que se deba definir una nueva operación en cualquier clase `Order`, se requiere modificar el código de dicha clase.

4.3 Enfoque de Diseño III (Patrón Composite)

Durante la discusión del patrón Composite, definimos operaciones sobre colecciones de objetos de una jerarquía de clases. Un Composite se diseña para mantener esta colección de objetos y operar sobre ella. Aplicando el patrón Composite, podemos definir una clase OrderComposite para definir operaciones sobre una colección de distintos objetos Order.

El atributo orderColl puede utilizarse para almacenar distintos tipos de objetos Order, y el método getOrderTotal itera sobre esta colección para recuperar los distintos montos de pedidos.

Este diseño no aborda completamente las limitaciones del Enfoque II. Permite definir una operación sobre una colección heterogénea, pero requiere modificar las clases de la jerarquía OrderComposite cada vez que se agrega una nueva operación.

4.4 Enfoque de Diseño IV (El Patrón Visitor)

Definamos una jerarquía de clases Order con una interfaz en la cima y tres implementadoras: CaliforniaOrder, NonCaliforniaOrder y OverseasOrder, cada una representando un tipo de pedido. La interfaz Order declara un método accept(OrderVisitor).

Cada implementadora de Order provee implementación del método accept(OrderVisitor). La VisitorInterface puede diseñarse como una interfaz Java que declare un conjunto de métodos visit(TipoDePedido), uno por cada clase en la jerarquía de pedidos. En otras palabras, estos métodos están diseñados para procesar distintos tipos de pedidos.

```
public interface VisitorInterface {  
    public void visit(NonCaliforniaOrder nco);  
    public void visit(CaliforniaOrder co);  
    public void visit(OverseasOrder oo);  
}
```

Definamos un visitante OrderVisitor como implementador de la VisitorInterface para calcular la suma de todos los totales de pedidos. Como parte de su implementación de los métodos de la VisitorInterface, el OrderVisitor recupera el monto del pedido, así como los importes adicionales por impuesto o por envío y manejo de distintos objetos Order, y los acumula en una variable de instancia privada orderTotal.

Flujo de la Aplicación

Al ejecutarse, el cliente OrderManager crea una instancia del OrderVisitor y muestra la interfaz de usuario necesaria para permitir al usuario crear distintos tipos de pedidos.

Cada vez que el usuario ingresa los datos de un pedido y hace clic en el botón Crear, el cliente OrderManager:

- Crea un objeto Order con los datos ingresados.
- Invoca el método accept(OrderVisitor) sobre el objeto Order, pasándole el objeto OrderVisitor. El objeto Order a su vez invoca el método visit del OrderVisitor, pasándose

a sí mismo como argumento. El OrderVisitor recupera el monto del pedido, el impuesto y los importes de envío usando los métodos públicos definidos por los distintos objetos Order, y los agrega de forma acumulativa al total en la variable de instancia privada orderTotal.

Cuando se hace clic en el botón Obtener Total, el cliente OrderManager invoca el método getOrderTotal del OrderVisitor. El OrderVisitor simplemente devuelve el valor almacenado en la variable de instancia orderTotal, representando el valor total de todos los pedidos creados.

5. Listados de Código

Listado 18.1 — Interfaz Order

```
public interface Order {  
    public void accept(OrderVisitor v);  
}
```

Listado 18.2 — Clase CaliforniaOrder

```
public class CaliforniaOrder implements Order {  
    private double orderAmount;  
    private double additionalTax;  
  
    public CaliforniaOrder() { }  
  
    public CaliforniaOrder(double inp_orderAmount,  
                           double inp_additionalTax) {  
        orderAmount = inp_orderAmount;  
        additionalTax = inp_additionalTax;  
    }  
  
    public double getOrderAmount() { return orderAmount; }  
    public double getAdditionalTax() { return additionalTax; }  
  
    public void accept(OrderVisitor v) {  
        v.visit(this);  
    }  
}
```

Listado 18.3 — Clase NonCaliforniaOrder

```
public class NonCaliforniaOrder implements Order {  
    private double orderAmount;  
  
    public NonCaliforniaOrder() { }  
  
    public NonCaliforniaOrder(double inp_orderAmount) {  
        orderAmount = inp_orderAmount;  
    }  
  
    public double getOrderAmount() { return orderAmount; }  
  
    public void accept(OrderVisitor v) {  
        v.visit(this);  
    }  
}
```

```
}  
}
```

Listado 18.4 — Clase OverseasOrder

```
public class OverseasOrder implements Order {  
    private double orderAmount;  
    private double additionalSH;  
  
    public OverseasOrder() { }  
  
    public OverseasOrder(double inp_orderAmount,  
                          double inp_additionalSH) {  
        orderAmount = inp_orderAmount;  
        additionalSH = inp_additionalSH;  
    }  
  
    public double getOrderAmount() { return orderAmount; }  
    public double getAdditionalSH() { return additionalSH; }  
  
    public void accept(OrderVisitor v) {  
        v.visit(this);  
    }  
}
```

Listado 18.5 — Clase OrderVisitor

```
class OrderVisitor implements VisitorInterface {  
    private Vector orderObjList;  
    private double orderTotal;  
  
    public OrderVisitor() {  
        orderObjList = new Vector();  
    }  
  
    public void visit(NonCaliforniaOrder inp_order) {  
        orderTotal = orderTotal + inp_order.getOrderAmount();  
    }  
  
    public void visit(CaliforniaOrder inp_order) {  
        orderTotal = orderTotal + inp_order.getOrderAmount() +  
            inp_order.getAdditionalTax();  
    }  
  
    public void visit(OverseasOrder inp_order) {  
        orderTotal = orderTotal + inp_order.getOrderAmount() +  
            inp_order.getAdditionalSH();  
    }  
  
    public double getOrderTotal() {  
        return orderTotal;  
    }  
}
```

Listado 18.6 — Clase OrderManager (fragmento)

```
public void actionPerformed(ActionEvent e) {  
    String totalResult = null;
```

```

        if (e.getActionCommand().equals(OrderManager.EXIT)) {
            System.exit(1);
        }
        if (e.getActionCommand().equals(OrderManager.CREATE_ORDER)) {
            // Obtener valores ingresados
            String orderType = objOrderManager.getOrderType();
            // ... (parseo y conversión de valores)
            Order order = createOrder(orderType, dblOrderAmount, dblTax, dblSH);
            OrderVisitor visitor = objOrderManager.getOrderVisitor();
            order.accept(visitor);
            objOrderManager.setTotalValue(" Pedido Creado Exitosamente");
        }
        if (e.getActionCommand().equals(OrderManager.GET_TOTAL)) {
            OrderVisitor visitor = objOrderManager.getOrderVisitor();
            totalResult = new Double(visitor.getOrderTotal()).toString();
            totalResult = " Total de Pedidos = " + totalResult;
            objOrderManager.setTotalValue(totalResult);
        }
    }

    public Order createOrder(String orderType,
        double orderAmount, double tax, double SH) {
        if (orderType.equalsIgnoreCase(OrderManager.CA_ORDER))
            return new CaliforniaOrder(orderAmount, tax);
        if (orderType.equalsIgnoreCase(OrderManager.NON_CA_ORDER))
            return new NonCaliforniaOrder(orderAmount);
        if (orderType.equalsIgnoreCase(OrderManager.OVERSEAS_ORDER))
            return new OverseasOrder(orderAmount, SH);
        return null;
    }
}

```

6. Definición de una Nueva Operación sobre la Colección

Definir una nueva operación sobre la colección de pedidos requiere crear un nuevo visitante. El nuevo visitante debe implementar la interfaz `VisitorInterface`, proveyendo implementación para los distintos métodos `visit(TipoDePedido)` necesarios para procesar los diferentes tipos de objetos `Order`. Con este diseño, no se requiere modificar ninguna clase existente de la colección.

7. Incorporación de un Nuevo Tipo de Pedido

Si se debe agregar un nuevo tipo de objeto a la colección — por ejemplo, una clase `DiscountOrder` que implemente la interfaz `Order` — entonces:

- Un método `visit(DiscountOrder)` correspondiente debe agregarse a la interfaz `VisitorInterface`.
- Este método debe ser implementado por la clase `OrderVisitor`.

8. Estructura de Clases del Diseño

Figura 18.5 — Jerarquía de Clases Order

«interface» Order			
accept(visitor: OrderVisitor)			
CaliforniaOrder	NonCaliforniaOrder	OverseasOrder	DiscountOrder
getOrderAmount(): double getAdditionalTax(): double accept(v: OrderVisitor)	getOrderAmount(): double accept(v: OrderVisitor)	getOrderAmount(): double getAdditionalSH(): double accept(v: OrderVisitor)	(ejemplo futuro)

Figura 18.6 — Estructura de Clases OrderVisitor

«interface» VisitorInterface
visit(nco: NonCaliforniaOrder) visit(co: CaliforniaOrder) visit(oo: OverseasOrder)
OrderVisitor
visit(nco: NonCaliforniaOrder) visit(co: CaliforniaOrder) visit(oo: OverseasOrder) getOrderTotal(): double

9. Preguntas de Práctica

1. Como parte de la discusión del patrón Composite, diseñamos una clase DirComponent. Rediseñe la clase DirComponent aplicando el patrón Visitor.
2. La sección de Preguntas de Práctica del patrón Composite discute tres aplicaciones que involucran operaciones sobre colecciones heterogéneas de objetos. Aplique el patrón Visitor para diseñar estas operaciones.